

IPL SQL Data Analysis

50 SQL Questions - Queries and Results in MySQL Workbench

Data Science - Final Practical Tasks
Data Science Instructor: Ananda Rimal

Database: ipl_db (MySQL) | Tables: matches (756 rows), deliveries (179,078 rows)
Relationship: matches.id (PRIMARY KEY) 1 ---- * deliveries.match_id (FOREIGN KEY)

Database Setup

Step 1 - Create the database and tables

The ipl_db database is created with two tables: matches (PRIMARY KEY id) and deliveries (PRIMARY KEY delivery_id, FOREIGN KEY match_id -> matches.id). DESCRIBE deliveries shows the PRI and MUL keys.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following SQL script:

```
-- Task 3 - IPL SQL Analysis
-- Step 1: Create the database and the two tables (matches, deliveries)
-- Relationship: matches.id (PK) 1 ---- * deliveries.match_id (FK)
-- Database: MySQL (run in MySQL Workbench)
-- =====
8 • DROP DATABASE IF EXISTS ipl_db;
9 • CREATE DATABASE ipl_db;
10 • USE ipl_db;
11
```

The Schemas pane on the left shows the 'sys' database. The Result Grid pane shows the results of the DESCRIBE queries:

Field	Type	Null	Key	Default	Extra
delivery_id	int	NO	PRI		auto_increment
match_id	int	NO	MUL		
inning	tinyint	NO			
batting_team	varchar(50)	NO			
bowling_team	varchar(50)	NO			
over	tinyint	NO			
ball	tinyint	NO			
batsman	varchar(50)	NO			
non_striker	varchar(50)	NO			
bowler	varchar(50)	NO			
is_super_over	tinyint	NO		0	
wide_runs	int	NO		0	
hits_run	int	NO		0	

The Output pane shows the results of the DESCRIBE queries:

#	Time	Action	Message	Duration / Fetch
126	12:21:45	DESCRIBE matches	18 row(s) returned	0.000 sec / 0.000 sec
127	12:21:45	DESCRIBE deliveries	22 row(s) returned	0.000 sec / 0.000 sec

Step 2 - Import data/matches.csv and data/deliveries.csv

Both CSV files are imported from the project data folder with LOAD DATA LOCAL INFILE. Result: 756 matches and 179,078 deliveries, 0 warnings.

The screenshot displays the MySQL Workbench interface. The 'Navigator' pane on the left shows the 'sys' schema. The 'SQL File 5*' pane shows the execution of a script with the following content:

```
1  -- =====
2  -- Task 3 - IPL SQL Analysis
3  -- Step 2: Import data/matches.csv and data/deliveries.csv into the tables
4  --
5  -- Requirements (one-time setup):
6  -- 1) Server: SET GLOBAL local_infile = 1;
7  -- 2) Workbench connection > Advanced > Others: OPT_LOCAL_INFILE=1
8  -- Update the two file paths below if the repository is in another folder.
9  -- =====
10 • USE ipl db;
```

The 'Result Grid' pane shows the following data:

matches_rows	deliveries_rows
756	179078

The 'Output' pane shows the following messages:

#	Time	Action	Message	Duration / Fetch
130	12:22:10	LOAD DATA LOCAL INFILE 'C:/Users/yugal/OneDrive/Desktop/IPL-DATA-ANALYSIS-TASK/data/deliveries...	179078 row(s) affected Records: 179078 Deleted: 0 Skipped: 0 Warnings: 0	6.859 sec
131	12:22:17	SELECT (SELECT COUNT(*) FROM matches) AS matches_rows, (SELECT COUNT(*) FROM deliveries...	1 row(s) returned	0.047 sec / 0.000 sec

The status bar at the bottom indicates 'Query Completed'.

Step 3 - Verify the primary keys and foreign key

information_schema.KEY_COLUMN_USAGE confirms both primary keys and the foreign key fk_deliveries_matches (deliveries.match_id -> matches.id).

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ipl_db' expanded, showing tables 'deliveries' and 'matches'. The main editor window contains a SQL query:
1 -- Primary key and foreign key defined on the two IPL tables
2 SELECT TABLE_NAME, COLUMN_NAME, CONSTRAINT_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
3 FROM information_schema.KEY_COLUMN_USAGE
4 WHERE TABLE_SCHEMA = 'ipl_db';
The 'Result Grid' shows the output of this query:

TABLE_NAME	COLUMN_NAME	CONSTRAINT_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
deliveries	delivery_id	PRIMARY		
deliveries	match_id	fk_deliveries_matches	matches	id
matches	id	PRIMARY		

The bottom panel shows the 'Output' tab with 'Action Output' selected. It displays two rows of execution results:

#	Time	Action	Message	Duration / Fetch
131	12:22:17	SELECT (SELECT COUNT(*) FROM matches) AS matches_rows, (SELECT COUNT(*) FROM deliveries...	1 row(s) returned	0.047 sec / 0.000 sec
132	12:23:11	SELECT TABLE_NAME, COLUMN_NAME, CONSTRAINT_NAME, REFERENCED_TABLE_NAME, REFERE...	3 row(s) returned	0.047 sec / 0.000 sec

The status bar at the bottom indicates 'Query Completed'.

Part A - Basic Queries and Filtering

Q1. Display the first 10 records from the matches table.

```
SELECT * FROM matches LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor at the top contains the query: `SELECT * FROM matches LIMIT 10;`. The left sidebar shows the 'SCHEMAS' panel with 'ipl_db' selected, containing tables like 'deliveries', 'matches', 'views', 'stored_procedures', and 'functions'. The 'Administration' tab is active, showing 'Schemas' and 'Information'. The 'Result Grid' at the bottom displays 10 rows of match data. The 'Output' panel at the bottom shows the execution log with two messages: '3 row(s) returned' and '10 row(s) returned'.

#	id	season	city	date	team1	team2	toss_winner	toss_decision	result	dl_applied	winner	win_by_runs	win_by_wickets	player_of_the_match
1	1	2017	Hyderabad	2017-04-05	Sunrisers Hyderabad	Royal Challengers Bangalore	Royal Challengers Bangalore	field	normal	0	Sunrisers Hyderabad	35	0	Yuvraj Singh
2	2	2017	Pune	2017-04-06	Mumbai Indians	Rising Pune Supergiant	Rising Pune Supergiant	field	normal	0	Rising Pune Supergiant	0	7	SPP
3	3	2017	Rajkot	2017-04-07	Gujarat Lions	Kolkata Knight Riders	Kolkata Knight Riders	field	normal	0	Kolkata Knight Riders	0	10	CA
4	4	2017	Indore	2017-04-08	Rising Pune Supergiant	Kings XI Punjab	Kings XI Punjab	field	normal	0	Kings XI Punjab	0	6	GJI
5	5	2017	Bangalore	2017-04-08	Royal Challengers Bangalore	Delhi Daredevils	Royal Challengers Bangalore	bat	normal	0	Royal Challengers Bangalore	15	0	KM
6	6	2017	Hyderabad	2017-04-09	Gujarat Lions	Sunrisers Hyderabad	Sunrisers Hyderabad	field	normal	0	Sunrisers Hyderabad	0	9	Rat
7	7	2017	Mumbai	2017-04-09	Kolkata Knight Riders	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	4	N R
8	8	2017	Indore	2017-04-10	Royal Challengers Bangalore	Kings XI Punjab	Royal Challengers Bangalore	bat	normal	0	Kings XI Punjab	0	8	AR
9	9	2017	Pune	2017-04-11	Delhi Daredevils	Rising Pune Supergiant	Rising Pune Supergiant	field	normal	0	Delhi Daredevils	97	0	SV
10	10	2017	Mumbai	2017-04-12	Sunrisers Hyderabad	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	4	JJE

Output Log:

#	Time	Action	Message	Duration / Fetch
132	12:23:11	SELECT TABLE_NAME, COLUMN_NAME, CONSTRAINT_NAME, REFERENCED_TABLE_NAME, REFERENC...	3 row(s) returned	0.047 sec / 0.000 sec
133	12:23:47	SELECT * FROM matches LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec

Q2. Display the first 10 records from the deliveries table.

```
SELECT * FROM deliveries LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor at the top contains the query: `SELECT * FROM deliveries LIMIT 10;`. The left sidebar shows the 'ipl_db' schema with tables 'deliveries' and 'matches'. The bottom panel displays the 'Result Grid' with 10 rows of data from the 'deliveries' table. The 'Output' panel at the bottom shows the execution log with two entries: a successful query execution at 12:23:47 and another at 12:23:58.

delivery_id	match_id	inning	batting_team	bowling_team	over	ball	batsman	non_striker	bowler	is_super_over	wide_runs	bye_runs	legbye_runs	noball_runs	penalty_runs	batsman
1	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	1	DA Warner	S Dhawan	TS Mills	0	0	0	0	0	0	0
2	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	2	DA Warner	S Dhawan	TS Mills	0	0	0	0	0	0	0
3	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	3	DA Warner	S Dhawan	TS Mills	0	0	0	0	0	0	4
4	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	4	DA Warner	S Dhawan	TS Mills	0	0	0	0	0	0	0
5	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	5	DA Warner	S Dhawan	TS Mills	0	2	0	0	0	0	0
6	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	6	S Dhawan	DA Warner	TS Mills	0	0	0	0	0	0	0
7	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	7	S Dhawan	DA Warner	TS Mills	0	0	0	1	0	0	0
8	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	2	1	S Dhawan	DA Warner	A Choudhary	0	0	0	0	0	0	1
9	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	2	2	DA Warner	S Dhawan	A Choudhary	0	0	0	0	0	0	4
10	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	2	3	DA Warner	S Dhawan	A Choudhary	0	0	0	0	1	0	0

deliveries 6 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
133	12:23:47	SELECT * FROM matches LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec
134	12:23:58	SELECT * FROM deliveries LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec

Query Completed

Q3. Count the total number of rows in the matches table.

```
SELECT COUNT(*) AS total_matches FROM matches;
```

MySQL Workbench interface showing the execution of a query. The query is: `SELECT COUNT(*) AS total_matches FROM matches;`

The result grid shows one row with the value 756 for the column `total_matches`.

The bottom panel shows the action output with two entries:

#	Time	Action	Message	Duration / Fetch
134	12:23:58	SELECT * FROM deliveries LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec
135	12:24:08	SELECT COUNT(*) AS total_matches FROM matches LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec

Q4. Count the total number of rows in the deliveries table.

```
SELECT COUNT(*) AS total_deliveries FROM deliveries;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q4. Count the total number of rows in the deliveries table.  
SELECT COUNT(*) AS total_deliveries FROM deliveries;
```

The Results window displays the output of the query:

total_deliveries
179078

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
135	12:24:08	SELECT COUNT(*) AS total_matches FROM matches LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
136	12:24:31	SELECT COUNT(*) AS total_deliveries FROM deliveries LIMIT 0, 1000	1 row(s) returned	0.032 sec / 0.000 sec

Q5. Display all unique IPL seasons from the matches table.

```
SELECT DISTINCT season FROM matches ORDER BY season;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the query: `-- Q5. Display all unique IPL seasons from the matches table.` followed by `SELECT DISTINCT season FROM matches ORDER BY season;`. The Results window displays the output of the query, showing a single column 'season' with 12 rows of data from 2008 to 2019. The Output window shows the execution log with two entries: a successful query execution (136) and a successful query execution (137) for the current query, both returning 1 row(s) and 12 row(s) respectively.

SQL Editor:

```
-- Q5. Display all unique IPL seasons from the matches table.
SELECT DISTINCT season FROM matches ORDER BY season;
```

Results:

season
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019

Output:

#	Time	Action	Message	Duration / Fetch
136	12:24:31	SELECT COUNT(*) AS total_deliveries FROM deliveries LIMIT 0, 1000	1 row(s) returned	0.032 sec / 0.000 sec
137	12:24:39	SELECT DISTINCT season FROM matches ORDER BY season LIMIT 0, 1000	12 row(s) returned	0.016 sec / 0.000 sec

Q6. Display all unique cities where IPL matches were played.

```
SELECT DISTINCT city FROM matches WHERE city IS NOT NULL ORDER BY city;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q6. Display all unique cities where IPL matches were played.  
SELECT DISTINCT city FROM matches WHERE city IS NOT NULL ORDER BY city;
```

The Results window displays the output of the query as a table with one column, 'city'. The cities listed are:

city
Abu Dhabi
Ahmedabad
Bangalore
Bengaluru
Bloemfontein
Cape Town
Centurion
Chandigarh
Chennai
Cuttack
Delhi
Dharamsala
Durban

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
137	12:24:39	SELECT DISTINCT season FROM matches ORDER BY season LIMIT 0, 1000	12 row(s) returned	0.016 sec / 0.000 sec
138	12:24:47	SELECT DISTINCT city FROM matches WHERE city IS NOT NULL ORDER BY city LIMIT 0, 1000	32 row(s) returned	0.000 sec / 0.000 sec

Q7. Display all unique team names appearing in the team1 column.

```
SELECT DISTINCT team1 FROM matches ORDER BY team1;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the query: `SELECT DISTINCT team1 FROM matches ORDER BY team1;`. The Results window displays the output of the query, showing a list of unique team names. The Output window shows the execution log, indicating that the query was executed successfully and returned 15 rows.

Result Grid

team1
Chennai Super Kings
Deccan Chargers
Delhi Capitals
Delhi Daredevils
Gujarat Lions
Kings XI Punjab
Kochi Tuskers Kerala
Kolkata Knight Riders
Mumbai Indians
Pune Warriors
Rajasthan Royals
Rising Pune Supergiant
Virat Kohli

Output

#	Time	Action	Message	Duration / Fetch
138	12:24:47	SELECT DISTINCT city FROM matches WHERE city IS NOT NULL ORDER BY city LIMIT 0, 1000	32 row(s) returned	0.000 sec / 0.000 sec
139	12:24:56	SELECT DISTINCT team1 FROM matches ORDER BY team1 LIMIT 0, 1000	15 row(s) returned	0.000 sec / 0.000 sec

Q8. Display all matches played in the 2017 season.

```
SELECT * FROM matches WHERE season = 2017;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the query: `-- Q8. Display all matches played in the 2017 season.` followed by `SELECT * FROM matches WHERE season = 2017;`. The query is executed, and the results are displayed in the Result Grid. The Result Grid shows 15 rows of match data, including columns for id, season, city, date, team1, team2, toss_winner, toss_decision, result, dl_applied, winner, win_by_runs, and win_by_wickets. The Output pane shows the execution log with two entries: 139 12:24:56 SELECT DISTINCT team1 FROM matches ORDER BY team1 LIMIT 0, 1000 (15 row(s) returned) and 140 12:25:04 SELECT * FROM matches WHERE season = 2017 LIMIT 0, 1000 (59 row(s) returned).

id	season	city	date	team1	team2	toss_winner	toss_decision	result	dl_applied	winner	win_by_runs	win_by_wickets
1	2017	Hyderabad	2017-04-05	Sunrisers Hyderabad	Royal Challengers Bangalore	Royal Challengers Bangalore	field	normal	0	Sunrisers Hyderabad	35	0
2	2017	Pune	2017-04-06	Mumbai Indians	Rising Pune Supergiant	Rising Pune Supergiant	field	normal	0	Rising Pune Supergiant	0	7
3	2017	Rajkot	2017-04-07	Gujarat Lions	Kolkata Knight Riders	Kolkata Knight Riders	field	normal	0	Kolkata Knight Riders	0	10
4	2017	Indore	2017-04-08	Rising Pune Supergiant	Kings XI Punjab	Kings XI Punjab	field	normal	0	Kings XI Punjab	0	6
5	2017	Bangalore	2017-04-08	Royal Challengers Bangalore	Delhi Daredevils	Royal Challengers Bangalore	bat	normal	0	Royal Challengers Bangalore	15	0
6	2017	Hyderabad	2017-04-09	Gujarat Lions	Sunrisers Hyderabad	Sunrisers Hyderabad	field	normal	0	Sunrisers Hyderabad	0	9
7	2017	Mumbai	2017-04-09	Kolkata Knight Riders	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	4
8	2017	Indore	2017-04-10	Royal Challengers Bangalore	Kings XI Punjab	Royal Challengers Bangalore	bat	normal	0	Kings XI Punjab	0	8
9	2017	Pune	2017-04-11	Delhi Daredevils	Rising Pune Supergiant	Rising Pune Supergiant	field	normal	0	Delhi Daredevils	97	0
10	2017	Mumbai	2017-04-12	Sunrisers Hyderabad	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	4
11	2017	Kolkata	2017-04-13	Kings XI Punjab	Kolkata Knight Riders	Kolkata Knight Riders	field	normal	0	Kolkata Knight Riders	0	8
12	2017	Bangalore	2017-04-14	Royal Challengers Bangalore	Mumbai Indians	Mumbai Indians	bat	normal	0	Mumbai Indians	0	4

Q9. Display all matches where the toss decision was field.

```
SELECT * FROM matches WHERE toss_decision = 'field';
```

The screenshot shows the MySQL Workbench interface. The 'Navigator' pane on the left displays the 'ipl_db' schema with tables 'deliveries' and 'matches'. The 'SQL File 5' editor shows the query: `-- Q9. Display all matches where the toss decision was field.` and `SELECT * FROM matches WHERE toss_decision = 'field';`. The 'Result Grid' pane displays 14 rows of match data. The 'Output' pane shows the execution log with two entries: a successful query execution at 12:26:01 returning 53 rows, and a successful query execution at 12:26:26 returning 463 rows.

	id	season	city	date	team1	team2	toss_winner	toss_decision	result	dl_applied	winner	win_by_runs	win_by_wickets	player
1	2017	Hyderabad	2017-04-05	Sunrisers Hyderabad	Royal Challengers Bangalore	Royal Challengers Bangalore	field	normal	0	0	Sunrisers Hyderabad	35	0	Yuvra
2	2017	Pune	2017-04-06	Mumbai Indians	Rising Pune Supergiant	Rising Pune Supergiant	field	normal	0	0	Rising Pune Supergiant	0	7	SPD S
3	2017	Rajkot	2017-04-07	Gujarat Lions	Kolkata Knight Riders	Kolkata Knight Riders	field	normal	0	0	Kolkata Knight Riders	0	10	CA Ly
4	2017	Indore	2017-04-08	Rising Pune Supergiant	Kings XI Punjab	Kings XI Punjab	field	normal	0	0	Kings XI Punjab	0	6	GJ Me
6	2017	Hyderabad	2017-04-09	Gujarat Lions	Sunrisers Hyderabad	Sunrisers Hyderabad	field	normal	0	0	Sunrisers Hyderabad	0	9	Rashi
7	2017	Mumbai	2017-04-09	Kolkata Knight Riders	Mumbai Indians	Mumbai Indians	field	normal	0	0	Mumbai Indians	0	4	N Rar
9	2017	Pune	2017-04-11	Delhi Daredevils	Rising Pune Supergiant	Rising Pune Supergiant	field	normal	0	0	Delhi Daredevils	97	0	SV Sa
10	2017	Mumbai	2017-04-12	Sunrisers Hyderabad	Mumbai Indians	Mumbai Indians	field	normal	0	0	Mumbai Indians	0	4	JJ Bu
11	2017	Kolkata	2017-04-13	Kings XI Punjab	Kolkata Knight Riders	Kolkata Knight Riders	field	normal	0	0	Kolkata Knight Riders	0	8	SP Ne
12	2017	Bangalore	2017-04-14	Royal Challengers Bangalore	Mumbai Indians	Mumbai Indians	field	normal	0	0	Mumbai Indians	0	4	KA Pc
13	2017	Rajkot	2017-04-14	Rising Pune Supergiant	Gujarat Lions	Gujarat Lions	field	normal	0	0	Gujarat Lions	0	7	AJ Ty
14	2017	Kolkata	2017-04-15	Kolkata Knight Riders	Sunrisers Hyderabad	Sunrisers Hyderabad	field	normal	0	0	Kolkata Knight Riders	17	0	DUT H

matches 16 x

Output

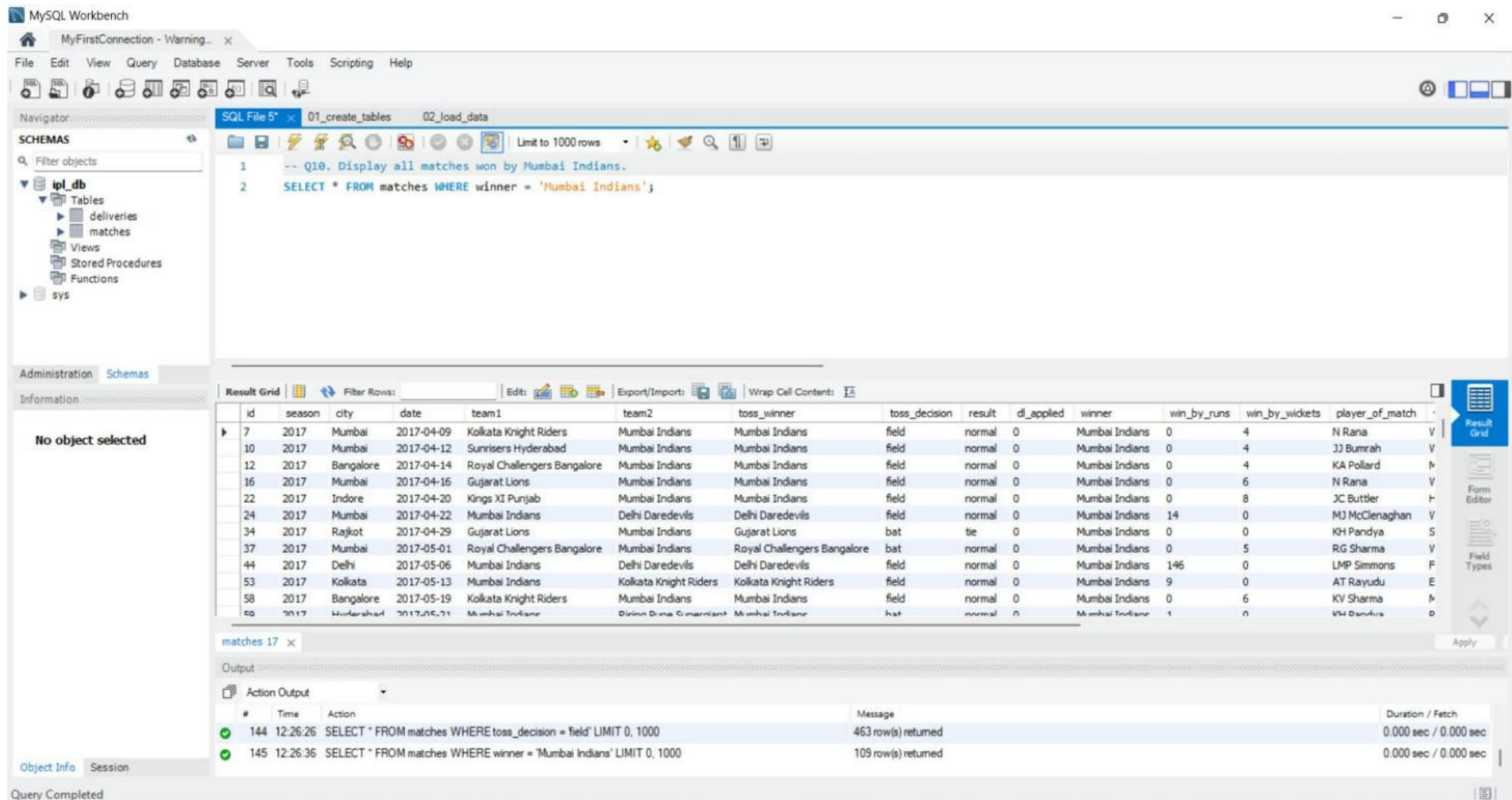
Action Output

#	Time	Action	Message	Duration / Fetch
143	12:26:01	SELECT * FROM matches WHERE win_by_runs > 50 LIMIT 0, 1000	53 row(s) returned	0.000 sec / 0.000 sec
144	12:26:26	SELECT * FROM matches WHERE toss_decision = 'field' LIMIT 0, 1000	463 row(s) returned	0.000 sec / 0.000 sec

Query Completed

Q10. Display all matches won by Mumbai Indians.

```
SELECT * FROM matches WHERE winner = 'Mumbai Indians';
```



MySQL Workbench

MyFirstConnection - Warning_ x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

ipl_db

Tables

deliveries

matches

Views

Stored Procedures

Functions

sys

Administration Schemas

Information

No object selected

SQL File 5 x 01_create_tables 02_load_data

Limit to 1000 rows

```
1 -- Q10. Display all matches won by Mumbai Indians.
2 SELECT * FROM matches WHERE winner = 'Mumbai Indians';
```

Result Grid

	id	season	city	date	team1	team2	toss_winner	toss_decision	result	dl_applied	winner	win_by_runs	win_by_wickets	player_of_match
7	2017	Mumbai	2017-04-09	Kolkata Knight Riders	Mumbai Indians	Mumbai Indians	field	normal	0	0	Mumbai Indians	0	4	N Rana
10	2017	Mumbai	2017-04-12	Sunrisers Hyderabad	Mumbai Indians	Mumbai Indians	field	normal	0	0	Mumbai Indians	0	4	JJ Bumrah
12	2017	Bangalore	2017-04-14	Royal Challengers Bangalore	Mumbai Indians	Mumbai Indians	field	normal	0	0	Mumbai Indians	0	4	KA Pollard
16	2017	Mumbai	2017-04-16	Gujarat Lions	Mumbai Indians	Mumbai Indians	field	normal	0	0	Mumbai Indians	0	6	N Rana
22	2017	Indore	2017-04-20	Kings XI Punjab	Mumbai Indians	Mumbai Indians	field	normal	0	0	Mumbai Indians	0	8	JC Buttler
24	2017	Mumbai	2017-04-22	Mumbai Indians	Delhi Daredevils	Delhi Daredevils	field	normal	0	0	Mumbai Indians	14	0	MJ McClenaghan
34	2017	Rajkot	2017-04-29	Gujarat Lions	Mumbai Indians	Gujarat Lions	bat	tie	0	0	Mumbai Indians	0	0	KH Pandya
37	2017	Mumbai	2017-05-01	Royal Challengers Bangalore	Mumbai Indians	Royal Challengers Bangalore	bat	normal	0	0	Mumbai Indians	0	5	RG Sharma
44	2017	Delhi	2017-05-06	Mumbai Indians	Delhi Daredevils	Delhi Daredevils	field	normal	0	0	Mumbai Indians	146	0	LMP Simmons
53	2017	Kolkata	2017-05-13	Mumbai Indians	Kolkata Knight Riders	Kolkata Knight Riders	field	normal	0	0	Mumbai Indians	9	0	AT Rayudu
58	2017	Bangalore	2017-05-19	Kolkata Knight Riders	Mumbai Indians	Mumbai Indians	field	normal	0	0	Mumbai Indians	0	6	KV Sharma
60	2017	Mumbai	2017-05-21	Mumbai Indians	Delhi Daredevils	Mumbai Indians	bat	normal	0	0	Mumbai Indians	1	0	VH Dandekar

matches 17 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
144	12:26:26	SELECT * FROM matches WHERE toss_decision = 'field' LIMIT 0, 1000	463 row(s) returned	0.000 sec / 0.000 sec
145	12:26:36	SELECT * FROM matches WHERE winner = 'Mumbai Indians' LIMIT 0, 1000	109 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Query Completed

Q11. Display matches where win_by_runs is greater than 50.

```
SELECT * FROM matches WHERE win_by_runs > 50;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the query: `-- Q11. Display matches where win_by_runs is greater than 50.` and `SELECT * FROM matches WHERE win_by_runs > 50;`. The Results Grid displays 18 rows of match data. The Output pane shows the execution log with two entries: one for a query that returned 109 rows and another for the current query that returned 53 rows.

#	id	season	city	date	team1	team2	toss_winner	toss_decision	result	dl_applied	winner	win_by_runs	win_by_wickets
9	2017	Pune	2017-04-11	Delhi Daredevils	Rising Pune Supergiant	Rising Pune Supergiant	field	normal	0	0	Delhi Daredevils	97	0
15	2017	Delhi	2017-04-15	Delhi Daredevils	Kings XI Punjab	Delhi Daredevils	bat	normal	0	0	Delhi Daredevils	51	0
27	2017	Kolkata	2017-04-23	Kolkata Knight Riders	Royal Challengers Bangalore	Royal Challengers Bangalore	field	normal	0	0	Kolkata Knight Riders	82	0
33	2017	Pune	2017-04-29	Rising Pune Supergiant	Royal Challengers Bangalore	Royal Challengers Bangalore	field	normal	0	0	Rising Pune Supergiant	61	0
44	2017	Delhi	2017-05-06	Mumbai Indians	Delhi Daredevils	Delhi Daredevils	field	normal	0	0	Mumbai Indians	146	0
60	2008	Bangalore	2008-04-18	Kolkata Knight Riders	Royal Challengers Bangalore	Royal Challengers Bangalore	field	normal	0	0	Kolkata Knight Riders	140	0
69	2008	Chandigarh	2008-04-25	Kings XI Punjab	Mumbai Indians	Mumbai Indians	field	normal	0	0	Kings XI Punjab	66	0
101	2008	Jaipur	2008-05-17	Rajasthan Royals	Royal Challengers Bangalore	Royal Challengers Bangalore	field	normal	0	0	Rajasthan Royals	65	0
115	2008	Mumbai	2008-05-30	Rajasthan Royals	Delhi Daredevils	Delhi Daredevils	field	normal	0	0	Rajasthan Royals	105	0
119	2009	Cape Town	2009-04-18	Royal Challengers Bangalore	Rajasthan Royals	Royal Challengers Bangalore	bat	normal	0	0	Royal Challengers Bangalore	75	0
122	2009	Port Elizab...	2009-04-20	Chennai Super Kings	Royal Challengers Bangalore	Chennai Super Kings	bat	normal	0	0	Chennai Super Kings	92	0
137	2009	Port Elizab...	2009-04-27	Mumbai Indians	Kolkata Knight Riders	Mumbai Indians	bat	normal	0	0	Mumbai Indians	87	0

Output Log:

#	Time	Action	Message	Duration / Fetch
145	12:26:36	SELECT * FROM matches WHERE winner = 'Mumbai Indians' LIMIT 0, 1000	109 row(s) returned	0.000 sec / 0.000 sec
146	12:26:46	SELECT * FROM matches WHERE win_by_runs > 50 LIMIT 0, 1000	53 row(s) returned	0.015 sec / 0.000 sec

Q12. Display matches where win_by_wickets equals 10.

```
SELECT * FROM matches WHERE win_by_wickets = 10;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the query: `SELECT * FROM matches WHERE win_by_wickets = 10;`. The Results Grid displays 11 rows of match data where the winning team won by 10 wickets. The Output pane shows the execution of the query, indicating that 11 rows were returned.

#	id	season	city	date	team1	team2	toss_winner	toss_decision	result	dl_applied	winner	win_by_runs	win_by_wickets	player_of
3	2017	Rajkot	2017-04-07	Gujarat Lions	Kolkata Knight Riders	Kolkata Knight Riders	field	normal	0	0	Kolkata Knight Riders	0	10	CA Lynn
35	2017	Chandigarh	2017-04-30	Delhi Daredevils	Kings XI Punjab	Kings XI Punjab	field	normal	0	0	Kings XI Punjab	0	10	Sandeep S
72	2008	Mumbai	2008-04-27	Mumbai Indians	Deccan Chargers	Deccan Chargers	field	normal	0	0	Deccan Chargers	0	10	AC Gilchris
120	2009	Cape Town	2009-04-19	Kings XI Punjab	Delhi Daredevils	Delhi Daredevils	field	normal	1	0	Delhi Daredevils	0	10	DL Vettori
184	2010	Bangalore	2010-03-18	Rajasthan Royals	Royal Challengers Bangalore	Royal Challengers Bangalore	field	normal	0	0	Royal Challengers Bangalore	0	10	JH Kallis
299	2011	Mumbai	2011-05-20	Mumbai Indians	Rajasthan Royals	Mumbai Indians	bat	normal	0	0	Rajasthan Royals	0	10	SR Watson
377	2012	Jaipur	2012-05-20	Rajasthan Royals	Mumbai Indians	Rajasthan Royals	bat	normal	0	0	Mumbai Indians	0	10	DR Smith
391	2013	Chandigarh	2013-04-10	Kings XI Punjab	Chennai Super Kings	Chennai Super Kings	field	normal	0	0	Chennai Super Kings	0	10	MEK Husse
543	2015	Delhi	2015-04-26	Delhi Daredevils	Royal Challengers Bangalore	Royal Challengers Bangalore	field	normal	0	0	Royal Challengers Bangalore	0	10	VR Aaron
591	2016	Rajkot	2016-04-21	Gujarat Lions	Sunrisers Hyderabad	Sunrisers Hyderabad	field	normal	0	0	Sunrisers Hyderabad	0	10	B Kumar
7941	2018	Indore	2018-05-14	Kings XI Punjab	Royal Challengers Bangalore	Royal Challengers Bangalore	field	normal	0	0	Royal Challengers Bangalore	0	10	UT Yadav

#	Time	Action	Message	Duration / Fetch
146	12:26:46	SELECT * FROM matches WHERE win_by_runs > 50 LIMIT 0, 1000	53 row(s) returned	0.015 sec / 0.000 sec
147	12:26:56	SELECT * FROM matches WHERE win_by_wickets = 10 LIMIT 0, 1000	11 row(s) returned	0.000 sec / 0.000 sec

Q13. Display matches where the toss winner and match winner are the same team.

```
SELECT * FROM matches WHERE toss_winner = winner;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Q13. Display matches where the toss winner and match winner are the same team.
SELECT * FROM matches WHERE toss_winner = winner;
```

The query has been executed, and the results are displayed in the Result Grid. The table shows 16 rows of match data where the toss winner and match winner are the same team.

id	season	city	date	team1	team2	toss_winner	toss_decision	result	dl_applied	winner	win_by_runs	win_by_wickets	player
2	2017	Pune	2017-04-06	Mumbai Indians	Rising Pune Supergiant	Rising Pune Supergiant	field	normal	0	Rising Pune Supergiant	0	7	SPD S
3	2017	Rajkot	2017-04-07	Gujarat Lions	Kolkata Knight Riders	Kolkata Knight Riders	field	normal	0	Kolkata Knight Riders	0	10	CA Ly
4	2017	Indore	2017-04-08	Rising Pune Supergiant	Kings XI Punjab	Kings XI Punjab	field	normal	0	Kings XI Punjab	0	6	GJ Me
5	2017	Bangalore	2017-04-08	Royal Challengers Bangalore	Delhi Daredevils	Royal Challengers Bangalore	bat	normal	0	Royal Challengers Bangalore	15	0	KM Ja
6	2017	Hyderabad	2017-04-09	Gujarat Lions	Sunrisers Hyderabad	Sunrisers Hyderabad	field	normal	0	Sunrisers Hyderabad	0	9	Rashi
7	2017	Mumbai	2017-04-09	Kolkata Knight Riders	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	4	N Rar
10	2017	Mumbai	2017-04-12	Sunrisers Hyderabad	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	4	JJ Bu
11	2017	Kolkata	2017-04-13	Kings XI Punjab	Kolkata Knight Riders	Kolkata Knight Riders	field	normal	0	Kolkata Knight Riders	0	8	SP Ne
12	2017	Bangalore	2017-04-14	Royal Challengers Bangalore	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	4	KA Pc
13	2017	Rajkot	2017-04-14	Rising Pune Supergiant	Gujarat Lions	Gujarat Lions	field	normal	0	Gujarat Lions	0	7	AJ Ty
15	2017	Delhi	2017-04-15	Delhi Daredevils	Kings XI Punjab	Delhi Daredevils	bat	normal	0	Delhi Daredevils	51	0	CJ Ar
16	2017	Mumbai	2017-04-16	Gujarat Lions	Mumbai Indians	Mumbai Indians	bat	normal	0	Mumbai Indians	0	6	N D ar

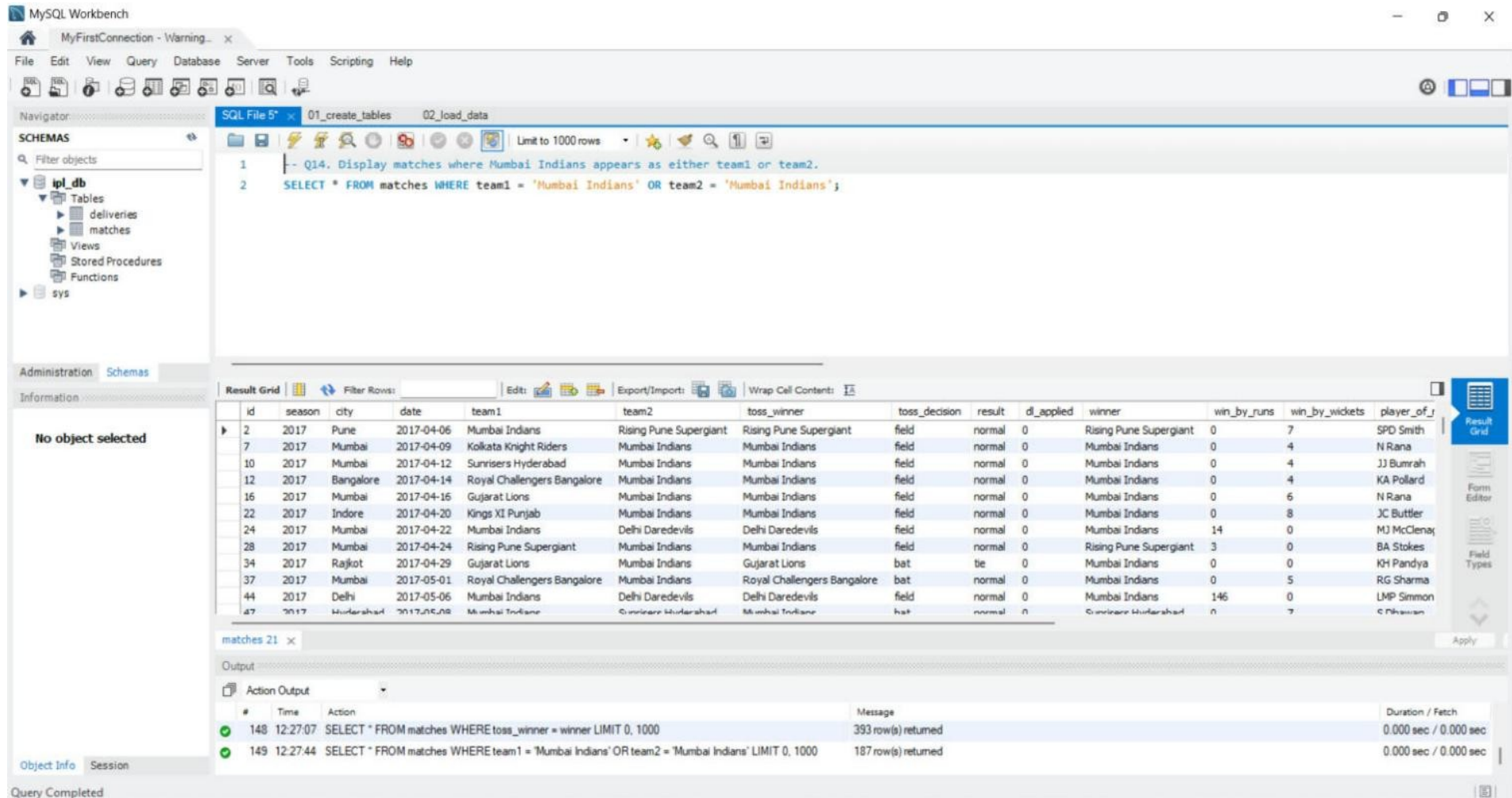
The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
147	12:26:56	SELECT * FROM matches WHERE win_by_wickets = 10 LIMIT 0, 1000	11 row(s) returned	0.000 sec / 0.000 sec
148	12:27:07	SELECT * FROM matches WHERE toss_winner = winner LIMIT 0, 1000	393 row(s) returned	0.000 sec / 0.000 sec

Query Completed

Q14. Display matches where Mumbai Indians appears as either team1 or team2.

```
SELECT * FROM matches WHERE team1 = 'Mumbai Indians' OR team2 = 'Mumbai Indians';
```



MySQL Workbench

MyFirstConnection - Warning_ x

File Edit View Query Database Server Tools Scripting Help

Navigator: 01_create_tables 02_load_data

SCHEMAS

Filter objects

ipl_db

- Tables
 - deliveries
 - matches
- Views
- Stored Procedures
- Functions

sys

Administration Schemas

Information

No object selected

Result Grid

id	season	city	date	team1	team2	toss_winner	toss_decision	result	dl_applied	winner	win_by_runs	win_by_wickets	player_of_the_match
2	2017	Pune	2017-04-06	Mumbai Indians	Rising Pune Supergiant	Rising Pune Supergiant	field	normal	0	Rising Pune Supergiant	0	7	SPD Smith
7	2017	Mumbai	2017-04-09	Kolkata Knight Riders	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	4	N Rana
10	2017	Mumbai	2017-04-12	Sunrisers Hyderabad	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	4	JJ Bumrah
12	2017	Bangalore	2017-04-14	Royal Challengers Bangalore	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	4	KA Pollard
16	2017	Mumbai	2017-04-16	Gujarat Lions	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	6	N Rana
22	2017	Indore	2017-04-20	Kings XI Punjab	Mumbai Indians	Mumbai Indians	field	normal	0	Mumbai Indians	0	8	JC Buttler
24	2017	Mumbai	2017-04-22	Mumbai Indians	Delhi Daredevils	Delhi Daredevils	field	normal	0	Mumbai Indians	14	0	MJ McClenay
28	2017	Mumbai	2017-04-24	Rising Pune Supergiant	Mumbai Indians	Mumbai Indians	field	normal	0	Rising Pune Supergiant	3	0	BA Stokes
34	2017	Rajkot	2017-04-29	Gujarat Lions	Mumbai Indians	Gujarat Lions	bat	tie	0	Mumbai Indians	0	0	KH Pandya
37	2017	Mumbai	2017-05-01	Royal Challengers Bangalore	Mumbai Indians	Royal Challengers Bangalore	bat	normal	0	Mumbai Indians	0	5	RG Sharma
44	2017	Delhi	2017-05-06	Mumbai Indians	Delhi Daredevils	Delhi Daredevils	field	normal	0	Mumbai Indians	146	0	LMP Simmon
47	2017	Hyderabad	2017-05-08	Mumbai Indians	Sunrisers Hyderabad	Mumbai Indians	bat	normal	0	Sunrisers Hyderabad	0	7	C Nkomo

matches 21 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
148	12:27:07	SELECT * FROM matches WHERE toss_winner = winner LIMIT 0, 1000	393 row(s) returned	0.000 sec / 0.000 sec
149	12:27:44	SELECT * FROM matches WHERE team1 = 'Mumbai Indians' OR team2 = 'Mumbai Indians' LIMIT 0, 1000	187 row(s) returned	0.000 sec / 0.000 sec

Query Completed

Q15. Display the 10 matches with the highest win_by_runs values.

```
SELECT id, season, team1, team2, winner, win_by_runs, venue
FROM matches ORDER BY win_by_runs DESC LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with 'ipl_db' selected, showing tables like 'deliveries', 'matches', 'views', 'stored_procedures', and 'functions'. The main editor window contains the following SQL query:

```
-- Q15. Display the 10 matches with the highest win_by_runs values.
SELECT id, season, team1, team2, winner, win_by_runs, venue
FROM matches ORDER BY win_by_runs DESC LIMIT 10;
```

The 'Result Grid' shows the top 10 matches based on 'win_by_runs' in descending order:

	id	season	team1	team2	winner	win_by_runs	venue
▶	44	2017	Mumbai Indians	Delhi Daredevils	Mumbai Indians	146	Feroz Shah Kotla
	620	2016	Royal Challengers Bangalore	Gujarat Lions	Royal Challengers Bangalore	144	M Chinnaswamy Stadium
	60	2008	Kolkata Knight Riders	Royal Challengers Bangalore	Kolkata Knight Riders	140	M Chinnaswamy Stadium
	557	2015	Royal Challengers Bangalore	Kings XI Punjab	Royal Challengers Bangalore	138	M Chinnaswamy Stadium
	411	2013	Royal Challengers Bangalore	Pune Warriors	Royal Challengers Bangalore	130	M Chinnaswamy Stadium
	11147	2019	Sunrisers Hyderabad	Royal Challengers Bangalore	Sunrisers Hyderabad	118	Rajiv Gandhi Intl. Cricket Stadium
	296	2011	Kings XI Punjab	Royal Challengers Bangalore	Kings XI Punjab	111	Himachal Pradesh Cricket Association Stadium
	115	2008	Rajasthan Royals	Delhi Daredevils	Rajasthan Royals	105	Wankhede Stadium
	7934	2018	Mumbai Indians	Kolkata Knight Riders	Mumbai Indians	102	Eden Gardens
	183	2010	Mumbai Indians	Delhi Daredevils	Mumbai Indians	98	Feroz Shah Kotla

The 'Output' panel at the bottom shows the execution log:

#	Time	Action	Message	Duration / Fetch
✓ 149	12:27:44	SELECT * FROM matches WHERE team1 = 'Mumbai Indians' OR team2 = 'Mumbai Indians' LIMIT 0, 1000	187 row(s) returned	0.000 sec / 0.000 sec
✓ 150	12:27:53	SELECT id, season, team1, team2, winner, win_by_runs, venue FROM matches ORDER BY win_by_runs DE...	10 row(s) returned	0.000 sec / 0.000 sec

Q16. Display the 10 matches with the highest win_by_wickets values.

```
SELECT id, season, team1, team2, winner, win_by_wickets, venue
FROM matches ORDER BY win_by_wickets DESC LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
1  -- Q16. Display the 10 matches with the highest win_by_wickets values.
2  SELECT id, season, team1, team2, winner, win_by_wickets, venue
3  FROM matches ORDER BY win_by_wickets DESC LIMIT 10;
```

The Results window displays the following data:

	id	season	team1	team2	winner	win_by_wickets	venue
72	2008	Mumbai Indians	Deccan Chargers	Deccan Chargers	10	Dr DY Patil Sports Academy	
377	2012	Rajasthan Royals	Mumbai Indians	Mumbai Indians	10	Sawai Mansingh Stadium	
3	2017	Gujarat Lions	Kolkata Knight Riders	Kolkata Knight Riders	10	Saurashtra Cricket Association Stadium	
299	2011	Mumbai Indians	Rajasthan Royals	Rajasthan Royals	10	Wankhede Stadium	
543	2015	Delhi Daredevils	Royal Challengers Bangalore	Royal Challengers Bangalore	10	Feroz Shah Kotla	
591	2016	Gujarat Lions	Sunrisers Hyderabad	Sunrisers Hyderabad	10	Saurashtra Cricket Association Stadium	
391	2013	Kings XI Punjab	Chennai Super Kings	Chennai Super Kings	10	Punjab Cricket Association Stadium, Mohali	
184	2010	Rajasthan Royals	Royal Challengers Bangalore	Royal Challengers Bangalore	10	M Chinnaswamy Stadium	
35	2017	Delhi Daredevils	Kings XI Punjab	Kings XI Punjab	10	Punjab Cricket Association IS Bindra Stadium, M...	
120	2009	Kings XI Punjab	Delhi Daredevils	Delhi Daredevils	10	Newlands	

The Output window shows the following action output:

#	Time	Action	Message	Duration / Fetch
150	12:27:53	SELECT id, season, team1, team2, winner, win_by_runs, venue FROM matches ORDER BY win_by_runs DE...	10 row(s) returned	0.000 sec / 0.000 sec
151	12:28:01	SELECT id, season, team1, team2, winner, win_by_wickets, venue FROM matches ORDER BY win_by_wicke...	10 row(s) returned	0.000 sec / 0.000 sec

Part B - Grouping and Aggregation

Q17. Count the number of matches played in each season.

```
SELECT season, COUNT(*) AS matches_played
FROM matches GROUP BY season ORDER BY season;
```

The screenshot displays the MySQL Workbench interface. The 'Navigator' pane on the left shows the 'ipl_db' database with tables 'deliveries' and 'matches'. The 'SQL File 5' editor contains the following query:

```
-- Q17. Count the number of matches played in each season.
SELECT season, COUNT(*) AS matches_played
FROM matches GROUP BY season ORDER BY season;
```

The 'Result Grid' pane shows the query results:

season	matches_played
2008	58
2009	57
2010	60
2011	73
2012	74
2013	76
2014	60
2015	59
2016	60
2017	59
2018	60
2019	60

The 'Output' pane at the bottom shows the execution log with two entries:

#	Time	Action	Message	Duration / Fetch
151	12:28:01	SELECT id, season, team1, team2, winner, win_by_wickets, venue FROM matches ORDER BY win_by_wicke...	10 row(s) returned	0.000 sec / 0.000 sec
152	12:28:11	SELECT season, COUNT(*) AS matches_played FROM matches GROUP BY season ORDER BY season LIM...	12 row(s) returned	0.000 sec / 0.000 sec

The status bar at the bottom indicates 'Query Completed'.

Q18. Count the number of matches played in each city.

```
SELECT city, COUNT(*) AS matches_played
FROM matches GROUP BY city ORDER BY matches_played DESC;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
1  -- Q18. Count the number of matches played in each city.
2  SELECT city, COUNT(*) AS matches_played
3  FROM matches GROUP BY city ORDER BY matches_played DESC;
```

The Results window displays the output of the query:

city	matches_played
Mumbai	101
Kolkata	77
Delhi	74
Bangalore	66
Hyderabad	64
Chennai	57
Jaipur	47
Chandigarh	46
Pune	38
Durban	15
Bengaluru	14
Visakhapatnam	13
Centurion	12

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
✓ 152	12:28:11	SELECT season, COUNT(*) AS matches_played FROM matches GROUP BY season ORDER BY season LIMIT...	12 row(s) returned	0.000 sec / 0.000 sec
✓ 153	12:28:21	SELECT city, COUNT(*) AS matches_played FROM matches GROUP BY city ORDER BY matches_played D...	33 row(s) returned	0.000 sec / 0.000 sec

Q19. Count the number of matches played at each venue.

```
SELECT venue, COUNT(*) AS matches_played
FROM matches GROUP BY venue ORDER BY matches_played DESC;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor at the top contains the following query:

```
-- Q19. Count the number of matches played at each venue.
SELECT venue, COUNT(*) AS matches_played
FROM matches GROUP BY venue ORDER BY matches_played DESC;
```

The Results window below the editor displays the output of the query as a table with two columns: **venue** and **matches_played**. The results are sorted in descending order of matches played.

venue	matches_played
Eden Gardens	77
M Chinnaswamy Stadium	73
Wankhede Stadium	73
Feroz Shah Kotla	67
Rajiv Gandhi International Stadium, Uppal	56
MA Chidambaram Stadium, Chepauk	49
Sawai Mansingh Stadium	47
Punjab Cricket Association Stadium, Mohali	35
Maharashtra Cricket Association Stadium	21
Dr DY Patil Sports Academy	17
Subrata Roy Sahara Stadium	17
Kingsmead	15
Dunelm Cricket Association SC Rendra Starl	14

The bottom of the interface shows the **Action Output** pane, which lists the execution of the query (Query 154) and its duration.

#	Time	Action	Message	Duration / Fetch
153	12:28:21	SELECT city, COUNT(*) AS matches_played FROM matches GROUP BY city ORDER BY matches_played D...	33 row(s) returned	0.000 sec / 0.000 sec
154	12:28:45	SELECT venue, COUNT(*) AS matches_played FROM matches GROUP BY venue ORDER BY matches_play...	41 row(s) returned	0.000 sec / 0.000 sec

Q20. Count the number of matches won by each team.

```
SELECT winner, COUNT(*) AS matches_won
FROM matches WHERE winner IS NOT NULL
GROUP BY winner ORDER BY matches_won DESC;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Q20. Count the number of matches won by each team.
SELECT winner, COUNT(*) AS matches_won
FROM matches WHERE winner IS NOT NULL
GROUP BY winner ORDER BY matches_won DESC;
```

The Results tab displays the output of the query in a table format:

winner	matches_won
Mumbai Indians	109
Chennai Super Kings	100
Kolkata Knight Riders	92
Royal Challengers Bangalore	84
Kings XI Punjab	82
Rajasthan Royals	75
Delhi Daredevils	67
Sunrisers Hyderabad	58
Deccan Chargers	29
Gujarat Lions	13
Pune Warriors	12
Rising Pune Supergiant	10
Delhi Capitals	10

The bottom panel shows the Output tab with the following log entries:

#	Time	Action	Message	Duration / Fetch
154	12:28:45	SELECT venue, COUNT(*) AS matches_played FROM matches GROUP BY venue ORDER BY matches_play...	41 row(s) returned	0.000 sec / 0.000 sec
155	12:28:54	SELECT winner, COUNT(*) AS matches_won FROM matches WHERE winner IS NOT NULL GROUP BY win...	15 row(s) returned	0.000 sec / 0.000 sec

The status bar at the bottom indicates "Query Completed".

Q21. Count the number of times each team won the toss.

```
SELECT toss_winner, COUNT(*) AS toss_wins
FROM matches GROUP BY toss_winner ORDER BY toss_wins DESC;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor at the top contains the following query:

```
-- Q21. Count the number of times each team won the toss.
SELECT toss_winner, COUNT(*) AS toss_wins
FROM matches GROUP BY toss_winner ORDER BY toss_wins DESC;
```

The left sidebar shows the 'SCHEMAS' panel with 'ipl_db' selected. The 'Administration' tab is active, and the 'Schemas' section shows 'No object selected'.

The 'Result Grid' at the bottom displays the results of the query. The table has two columns: 'toss_winner' and 'toss_wins'. The results are sorted in descending order of 'toss_wins'.

toss_winner	toss_wins
Mumbai Indians	98
Kolkata Knight Riders	92
Chennai Super Kings	89
Royal Challengers Bangalore	81
Kings XI Punjab	81
Delhi Daredevils	80
Rajasthan Royals	80
Sunrisers Hyderabad	46
Deccan Chargers	43
Pune Warriors	20
Gujarat Lions	15
Delhi Capitals	10
Kochi Tuskan Kerala	0

The 'Output' panel at the bottom shows the execution log. The last two actions are highlighted:

#	Time	Action	Message	Duration / Fetch
155	12:28:54	SELECT winner, COUNT(*) AS matches_won FROM matches WHERE winner IS NOT NULL GROUP BY win...	15 row(s) returned	0.000 sec / 0.000 sec
156	12:29:05	SELECT toss_winner, COUNT(*) AS toss_wins FROM matches GROUP BY toss_winner ORDER BY toss_wi...	15 row(s) returned	0.000 sec / 0.000 sec

Q22. Count the number of bat and field toss decisions.

```
SELECT toss_decision, COUNT(*) AS decisions
FROM matches GROUP BY toss_decision;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
1  -- Q22. Count the number of bat and field toss decisions.
2  SELECT toss_decision, COUNT(*) AS decisions
3  FROM matches GROUP BY toss_decision;
```

The Results window displays the output of the query in a table format:

toss_decision	decisions
field	463
bat	293

The bottom panel shows the Action Output log with the following entries:

#	Time	Action	Message	Duration / Fetch
156	12:29:05	SELECT toss_winner, COUNT(*) AS toss_wins FROM matches GROUP BY toss_winner ORDER BY toss_wi...	15 row(s) returned	0.000 sec / 0.000 sec
157	12:29:16	SELECT toss_decision, COUNT(*) AS decisions FROM matches GROUP BY toss_decision LIMIT 0, 1000	2 row(s) returned	0.015 sec / 0.000 sec

Q23. Find the top 10 players with the most Player of the Match awards.

```
SELECT player_of_match, COUNT(*) AS awards
FROM matches WHERE player_of_match IS NOT NULL
GROUP BY player_of_match ORDER BY awards DESC LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The query editor at the top contains the following SQL query:

```
1  Q23. Find the top 10 players with the most Player of the Match awards.
2  SELECT player_of_match, COUNT(*) AS awards
3  FROM matches WHERE player_of_match IS NOT NULL
4  GROUP BY player_of_match ORDER BY awards DESC LIMIT 10;
```

The left sidebar shows the 'SCHEMAS' panel with a tree view of the database structure. The 'ipl_db' database is selected, showing tables like 'deliveries', 'matches', 'views', 'stored_procedures', and 'functions'.

The 'Result Grid' at the bottom displays the results of the query. The table has two columns: 'player_of_match' and 'awards'. The results are as follows:

player_of_match	awards
CH Gayle	21
AB de Villiers	20
RG Sharma	17
DA Warner	17
MS Dhoni	17
YK Pathan	16
SR Watson	15
SK Raina	14
G Gambhir	13
MEK Hussey	12

The bottom panel shows the 'Output' tab with a table of query execution details:

#	Time	Action	Message	Duration / Fetch
159	12:31:23	SELECT season, ROUND(AVG(win_by_runs), 2) AS avg_win_by_runs FROM matches GROUP BY season O...	12 row(s) returned	0.000 sec / 0.000 sec
160	12:32:35	SELECT player_of_match, COUNT(*) AS awards FROM matches WHERE player_of_match IS NOT NULL GR...	10 row(s) returned	0.000 sec / 0.000 sec

Q24. Find the average win_by_runs for each season.

```
SELECT season, ROUND(AVG(win_by_runs), 2) AS avg_win_by_runs
FROM matches GROUP BY season ORDER BY season;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q24. Find the average win_by_runs for each season.
SELECT season, ROUND(AVG(win_by_runs), 2) AS avg_win_by_runs
FROM matches GROUP BY season ORDER BY season;
```

The Results window displays the output of the query in a table format:

season	avg_win_by_runs
2008	12.16
2009	13.40
2010	16.27
2011	15.04
2012	12.97
2013	16.33
2014	10.73
2015	14.41
2016	11.27
2017	13.36
2018	11.25
2019	11.08

The bottom of the interface shows the 'Action Output' window with the following log entries:

#	Time	Action	Message	Duration / Fetch
158	12:29:25	SELECT player_of_match, COUNT(*) AS awards FROM matches WHERE player_of_match IS NOT NULL GR...	10 row(s) returned	0.000 sec / 0.000 sec
159	12:31:23	SELECT season, ROUND(AVG(win_by_runs), 2) AS avg_win_by_runs FROM matches GROUP BY season O...	12 row(s) returned	0.000 sec / 0.000 sec

Q25. Find the average win_by_wickets for each season.

```
SELECT season, ROUND(AVG(win_by_wickets), 2) AS avg_win_by_wickets
FROM matches GROUP BY season ORDER BY season;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q25. Find the average win_by_wickets for each season.
SELECT season, ROUND(AVG(win_by_wickets), 2) AS avg_win_by_wickets
FROM matches GROUP BY season ORDER BY season;
```

The Results window displays the output of the query in a table format:

season	avg_win_by_wickets
2008	3.81
2009	3.16
2010	3.17
2011	3.63
2012	3.26
2013	2.99
2014	3.75
2015	2.51
2016	4.07
2017	3.46
2018	3.10
2019	3.37

The bottom of the interface shows the 'Output' tab with the following log entries:

#	Time	Action	Message	Duration / Fetch
160	12:32:35	SELECT player_of_match, COUNT(*) AS awards FROM matches WHERE player_of_match IS NOT NULL GR...	10 row(s) returned	0.000 sec / 0.000 sec
161	12:32:50	SELECT season, ROUND(AVG(win_by_wickets), 2) AS avg_win_by_wickets FROM matches GROUP BY sea...	12 row(s) returned	0.000 sec / 0.000 sec

Q26. Find the maximum win_by_runs recorded in each season.

```
SELECT season, MAX(win_by_runs) AS max_win_by_runs
FROM matches GROUP BY season ORDER BY season;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q26. Find the maximum win_by_runs recorded in each season.
SELECT season, MAX(win_by_runs) AS max_win_by_runs
FROM matches GROUP BY season ORDER BY season;
```

The Results window displays the following data:

season	max_win_by_runs
2008	140
2009	92
2010	98
2011	111
2012	86
2013	130
2014	93
2015	138
2016	144
2017	146
2018	102
2019	118

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
161	12:32:50	SELECT season, ROUND(AVG(win_by_wickets), 2) AS avg_win_by_wickets FROM matches GROUP BY season ORDER BY season;	12 row(s) returned	0.000 sec / 0.000 sec
162	12:33:05	SELECT season, MAX(win_by_runs) AS max_win_by_runs FROM matches GROUP BY season ORDER BY season;	12 row(s) returned	0.000 sec / 0.000 sec

Q27. Find the maximum win_by_wickets recorded in each season.

```
SELECT season, MAX(win_by_wickets) AS max_win_by_wickets
FROM matches GROUP BY season ORDER BY season;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q27. Find the maximum win_by_wickets recorded in each season.
SELECT season, MAX(win_by_wickets) AS max_win_by_wickets
FROM matches GROUP BY season ORDER BY season;
```

The Results window displays the following data:

season	max_win_by_wickets
2008	10
2009	10
2010	10
2011	10
2012	10
2013	10
2014	9
2015	10
2016	10
2017	10
2018	10
2019	9

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
162	12:33:05	SELECT season, MAX(win_by_runs) AS max_win_by_runs FROM matches GROUP BY season ORDER BY se...	12 row(s) returned	0.000 sec / 0.000 sec
163	12:33:18	SELECT season, MAX(win_by_wickets) AS max_win_by_wickets FROM matches GROUP BY season ORDE...	12 row(s) returned	0.000 sec / 0.000 sec

Part C - Ball-by-Ball Analysis

Q28. Count the total number of sixes (batsman_runs = 6) in the deliveries table.

```
SELECT COUNT(*) AS total_sixes FROM deliveries WHERE batsman_runs = 6;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
1  -- Q28. Count the total number of sixes (batsman_runs = 6) in the deliveries table.
2  SELECT COUNT(*) AS total_sixes FROM deliveries WHERE batsman_runs = 6;
```

The Results window displays the output of the query in a table format:

total_sixes
8170

The Output window shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
168	12:34:40	SELECT batsman, SUM(batsman_runs) AS total_runs FROM deliveries GROUP BY batsman ORDER BY total...	10 row(s) returned	0.203 sec / 0.000 sec
169	12:35:20	SELECT COUNT(*) AS total_sixes FROM deliveries WHERE batsman_runs = 6 LIMIT 0, 1000	1 row(s) returned	0.047 sec / 0.000 sec

The status bar at the bottom indicates "Query Completed".

Q29. Count the total number of fours (batsman_runs = 4) in the deliveries table.

```
SELECT COUNT(*) AS total_fours FROM deliveries WHERE batsman_runs = 4;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q29. Count the total number of fours (batsman_runs = 4) in the deliveries table.  
SELECT COUNT(*) AS total_fours FROM deliveries WHERE batsman_runs = 4;
```

The Results window displays the output of the query in a table format:

total_fours
20392

The Output window shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
171	12:35:43	SELECT SUM(wide_runs) AS total_wide_runs FROM deliveries LIMIT 0, 1000	1 row(s) returned	0.063 sec / 0.000 sec
172	12:36:23	SELECT COUNT(*) AS total_fours FROM deliveries WHERE batsman_runs = 4 LIMIT 0, 1000	1 row(s) returned	0.047 sec / 0.000 sec

Q30. Find the total number of wide runs in the deliveries table.

```
SELECT SUM(wide_runs) AS total_wide_runs FROM deliveries;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view of the 'ipl_db' database, including tables 'deliveries' and 'matches', and views 'Stored Procedures' and 'Functions'. The main editor window shows a SQL file with two lines of code: a comment and a query. The query is: `SELECT SUM(wide_runs) AS total_wide_runs FROM deliveries;`. The bottom panel shows the 'Result Grid' with one row: `total_wide_runs` with the value `6576`. The 'Output' panel shows the execution log with two entries: a successful query execution at 12:35:30 and a successful query execution at 12:35:43.

MySQL Workbench

MyFirstConnection - Warning_ x

File Edit View Query Database Server Tools Scripting Help

Navigator SQL File 5* 01_create_tables 02_load_data

Limit to 1000 rows

1 -- Q30. Find the total number of wide runs in the deliveries table.

2 SELECT SUM(wide_runs) AS total_wide_runs FROM deliveries;

Administration Schemas

Information

No object selected

Result Grid

total_wide_runs

6576

Result 43 x

Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
170	12:35:30	SELECT COUNT(*) AS total_fours FROM deliveries WHERE batsman_runs = 4 LIMIT 0, 1000	1 row(s) returned	0.063 sec / 0.000 sec
171	12:35:43	SELECT SUM(wide_runs) AS total_wide_runs FROM deliveries LIMIT 0, 1000	1 row(s) returned	0.063 sec / 0.000 sec

Object Info Session

Query Completed

Q31. Find the total number of no-ball runs in the deliveries table.

```
SELECT SUM(noball_runs) AS total_noball_runs FROM deliveries;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view of the 'ipl_db' database, including tables like 'deliveries' and 'matches'. The main editor window shows a SQL file with two queries. The second query is selected and executed. The 'Result Grid' at the bottom shows a single row with the value '749' for the column 'total_noball_runs'. The 'Output' panel at the bottom shows the execution log with two entries: a successful query execution and a message indicating that 756 rows were returned from a previous query.

MySQL Workbench

MyFirstConnection - Warning_ x

File Edit View Query Database Server Tools Scripting Help

Navigator SQL File 5* 01_create_tables 02_load_data

Limit to 1000 rows

1 |-- Q31. Find the total number of no-ball runs in the deliveries table.

2 • SELECT SUM(noball_runs) AS total_noball_runs FROM deliveries;

Administration Schemas

Information No object selected

Result Grid Filter Rows: Exports Wrap Cell Contents

total_noball_runs
749

Result 64 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
191	12:44:21	SELECT m.id AS match_id, m.season, m.winner, SUM(d.total_runs) AS match_total_runs FROM matches m JO...	756 row(s) returned	0.593 sec / 0.000 sec
192	12:45:19	SELECT SUM(noball_runs) AS total_noball_runs FROM deliveries LIMIT 0, 1000	1 row(s) returned	0.047 sec / 0.000 sec

Object Info Session

Query Completed

Q32. Find the total batsman runs scored by each batsman and show the top 10 run scorers.

```
SELECT batsman, SUM(batsman_runs) AS total_runs  
FROM deliveries GROUP BY batsman ORDER BY total_runs DESC LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q32. Find the total batsman runs scored by each batsman and show the top 10 run scorers.  
SELECT batsman, SUM(batsman_runs) AS total_runs  
FROM deliveries GROUP BY batsman ORDER BY total_runs DESC LIMIT 10;
```

The Results window displays the following data:

batsman	total_runs
V Kohli	5434
SK Raina	5415
RG Sharma	4914
DA Warner	4741
S Dhawan	4632
CH Gayle	4560
MS Dhoni	4477
RV Uthappa	4446
AB de Villiers	4428
G Gambhir	4223

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
167	12:34:27	SELECT SUM(noball_runs) AS total_noball_runs FROM deliveries LIMIT 0, 1000	1 row(s) returned	0.063 sec / 0.000 sec
168	12:34:40	SELECT batsman, SUM(batsman_runs) AS total_runs FROM deliveries GROUP BY batsman ORDER BY total...	10 row(s) returned	0.203 sec / 0.000 sec

Q33. Find the number of sixes hit by each batsman and show the top 10 six hitters.

```
SELECT batsman, COUNT(*) AS sixes
FROM deliveries WHERE batsman_runs = 6
GROUP BY batsman ORDER BY sixes DESC LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q33. Find the number of sixes hit by each batsman and show the top 10 six hitters.
SELECT batsman, COUNT(*) AS sixes
FROM deliveries WHERE batsman_runs = 6
GROUP BY batsman ORDER BY sixes DESC LIMIT 10;
```

The Results tab displays the following table:

batsman	sixes
CH Gayle	327
AB de Villiers	214
MS Dhoni	207
SK Raina	195
RG Sharma	194
V Kohli	191
DA Warner	181
SR Watson	177
KA Pollard	175
YK Pathan	161

The Output tab shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
173	12:36:42	SELECT SUM(noball_runs) AS total_noball_runs FROM deliveries LIMIT 0, 1000	1 row(s) returned	0.079 sec / 0.000 sec
174	12:36:52	SELECT batsman, COUNT(*) AS sixes FROM deliveries WHERE batsman_runs = 6 GROUP BY batsman ORD...	10 row(s) returned	0.094 sec / 0.000 sec

Q34. Find the number of fours hit by each batsman and show the top 10 four hitters.

```
SELECT batsman, COUNT(*) AS fours
FROM deliveries WHERE batsman_runs = 4
GROUP BY batsman ORDER BY fours DESC LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
1  -- Q34. Find the number of fours hit by each batsman and show the top 10 four hitters.
2  SELECT batsman, COUNT(*) AS fours
3  FROM deliveries WHERE batsman_runs = 4
4  GROUP BY batsman ORDER BY fours DESC LIMIT 10;
```

The Results pane displays the following table:

batsman	fours
S Dhawan	526
SK Raina	495
G Gambhir	492
V Kohli	482
DA Warner	459
RV Uthappa	436
RG Sharma	431
AM Rahane	405
CH Gayle	376
PA Patel	366

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
174	12:36:52	SELECT batsman, COUNT(*) AS sixes FROM deliveries WHERE batsman_runs = 6 GROUP BY batsman ORD...	10 row(s) returned	0.094 sec / 0.000 sec
175	12:37:15	SELECT batsman, COUNT(*) AS fours FROM deliveries WHERE batsman_runs = 4 GROUP BY batsman ORD...	10 row(s) returned	0.093 sec / 0.000 sec

Q35. Find the total runs scored by each batting team.

```
SELECT batting_team, SUM(total_runs) AS team_runs  
FROM deliveries GROUP BY batting_team ORDER BY team_runs DESC;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
1 -- Q35. Find the total runs scored by each batting team.  
2 SELECT batting_team, SUM(total_runs) AS team_runs  
3 FROM deliveries GROUP BY batting_team ORDER BY team_runs DESC;
```

The Results window displays the output of the query in a table format:

batting_team	team_runs
Mumbai Indians	29809
Royal Challengers Bangalore	28126
Kings XI Punjab	27893
Kolkata Knight Riders	27419
Chennai Super Kings	26418
Delhi Daredevils	24388
Rajasthan Royals	22431
Sunrisers Hyderabad	17059
Deccan Chargers	11463
Pune Warriors	6358
Gujarat Lions	4862
Delhi Capitals	2630
Delhi Daredevils	24388

The bottom of the interface shows the Action Output window with the following entries:

#	Time	Action	Message	Duration / Fetch
175	12:37:15	SELECT batsman, COUNT(*) AS fours FROM deliveries WHERE batsman_runs = 4 GROUP BY batsman ORD...	10 row(s) returned	0.093 sec / 0.000 sec
176	12:37:35	SELECT batting_team, SUM(total_runs) AS team_runs FROM deliveries GROUP BY batting_team ORDER BY ...	15 row(s) returned	0.219 sec / 0.000 sec

Q36. Find the total extra runs received by each batting team.

```
SELECT batting_team, SUM(extra_runs) AS extras_received  
FROM deliveries GROUP BY batting_team ORDER BY extras_received DESC;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q36. Find the total extra runs received by each batting team.  
SELECT batting_team, SUM(extra_runs) AS extras_received  
FROM deliveries GROUP BY batting_team ORDER BY extras_received DESC;
```

The Results window displays the output of the query, showing 15 rows of data. The columns are 'batting_team' and 'extras_received'.

batting_team	extras_received
Mumbai Indians	1645
Kolkata Knight Riders	1524
Kings XI Punjab	1425
Royal Challengers Bangalore	1351
Chennai Super Kings	1314
Delhi Daredevils	1273
Rajasthan Royals	1090
Sunrisers Hyderabad	809
Deccan Chargers	578
Pune Warriors	318
Gujarat Lions	233
Kochi Tuskers Kerala	143
Dinamo Duna Samaritan	101

The Output window shows the execution log with two entries:

#	Time	Action	Message	Duration / Fetch
176	12:37:35	SELECT batting_team, SUM(total_runs) AS team_runs FROM deliveries GROUP BY batting_team ORDER BY ...	15 row(s) returned	0.219 sec / 0.000 sec
177	12:37:56	SELECT batting_team, SUM(extra_runs) AS extras_received FROM deliveries GROUP BY batting_team ORD...	15 row(s) returned	0.219 sec / 0.000 sec

Q37. Count the number of deliveries bowled by each bowler and show the top 10.

```
SELECT bowler, COUNT(*) AS deliveries_bowled
FROM deliveries GROUP BY bowler ORDER BY deliveries_bowled DESC LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
1  -- Q37. Count the number of deliveries bowled by each bowler and show the top 10.
2  SELECT bowler, COUNT(*) AS deliveries_bowled
3  FROM deliveries GROUP BY bowler ORDER BY deliveries_bowled DESC LIMIT 10;
```

The Results window displays the following data:

bowler	deliveries_bowled
Harbhajan Singh	3451
A Mishra	3172
PP Chawla	3157
R Ashwin	3016
SL Malinga	2974
DJ Bravo	2711
B Kumar	2707
P Kumar	2637
UT Yadav	2605
SP Narine	2600

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
177	12:37:56	SELECT batting_team, SUM(extra_runs) AS extras_received FROM deliveries GROUP BY batting_team ORD...	15 row(s) returned	0.219 sec / 0.000 sec
178	12:38:18	SELECT bowler, COUNT(*) AS deliveries_bowled FROM deliveries GROUP BY bowler ORDER BY deliveries_...	10 row(s) returned	0.188 sec / 0.000 sec

Q38. Find the total runs conceded by each bowler using total_runs and show the top 10 highest totals.

```
SELECT bowler, SUM(total_runs) AS runs_conceded
FROM deliveries GROUP BY bowler ORDER BY runs_conceded DESC LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Q38. Find the total runs conceded by each bowler using total_runs and show the top 10 highest totals.
SELECT bowler, SUM(total_runs) AS runs_conceded
FROM deliveries GROUP BY bowler ORDER BY runs_conceded DESC LIMIT 10;
```

The query has been executed, and the results are displayed in the Result Grid. The results show the top 10 bowlers by total runs conceded:

bowler	runs_conceded
PP Chawla	4153
Harbhajan Singh	4050
A Mishra	3850
DJ Bravo	3733
UT Yadav	3640
SL Malinga	3511
R Ashwin	3391
P Kumar	3342
B Kumar	3264
RA Jadeja	3221

The bottom panel shows the Action Output, indicating that the query was executed successfully and returned 10 rows.

Q39. Count the number of recorded dismissals for each bowler, excluding run out, and show the top 10.

```
SELECT bowler, COUNT(*) AS dismissals
FROM deliveries
WHERE player_dismissed IS NOT NULL AND dismissal_kind <> 'run out'
GROUP BY bowler ORDER BY dismissals DESC LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor at the top contains the query for Q39. The left sidebar shows the 'ipl_db' schema with tables 'deliveries' and 'matches'. The bottom pane displays the 'Result Grid' with 10 rows of bowler names and their dismissal counts, sorted in descending order. The 'Output' pane at the bottom shows the execution log for two queries, with the second query (Q39) returning 10 rows.

Result Grid:

bowler	dismissals
SL Malinga	170
A Mishra	156
Harbhajan Singh	150
PP Chawla	149
DJ Bravo	147
B Kumar	133
R Ashwin	125
SP Narine	122
UT Yadav	119
RA Jadeja	108

Output:

#	Time	Action	Message	Duration / Fetch
179	12:38:39	SELECT bowler, SUM(total_runs) AS runs_conceded FROM deliveries GROUP BY bowler ORDER BY runs_c...	10 row(s) returned	0.219 sec / 0.000 sec
180	12:39:01	SELECT bowler, COUNT(*) AS dismissals FROM deliveries WHERE player_dismissed IS NOT NULL AND dismiss...	10 row(s) returned	0.187 sec / 0.000 sec

Q40. Count how many dismissals occurred for each dismissal_kind.

```
SELECT dismissal_kind, COUNT(*) AS dismissals
FROM deliveries WHERE dismissal_kind IS NOT NULL
GROUP BY dismissal_kind ORDER BY dismissals DESC;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q40. Count how many dismissals occurred for each dismissal_kind.
SELECT dismissal_kind, COUNT(*) AS dismissals
FROM deliveries WHERE dismissal_kind IS NOT NULL
GROUP BY dismissal_kind ORDER BY dismissals DESC;
```

The Results window displays the following data:

dismissal_kind	dismissals
caught	5348
bowled	1581
run out	852
lbw	540
stumped	278
caught and bowled	211
retired hurt	12
hit wicket	10
obstructing the field	2

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
192	12:45:19	SELECT SUM(noball_runs) AS total_noball_runs FROM deliveries LIMIT 0, 1000	1 row(s) returned	0.047 sec / 0.000 sec
193	12:45:50	SELECT dismissal_kind, COUNT(*) AS dismissals FROM deliveries WHERE dismissal_kind IS NOT NULL GR...	9 row(s) returned	0.062 sec / 0.000 sec

Part D - Analysis Using Both Tables

Q41. Join matches and deliveries using matches.id = deliveries.match_id and display 20 joined records.

```
SELECT m.id, m.season, m.venue, d.inning, d.batting_team, d.bowling_team,  
       d.`over`, d.ball, d.batsman, d.bowler, d.total_runs  
FROM matches m  
JOIN deliveries d ON m.id = d.match_id  
LIMIT 20;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q41. Join matches and deliveries using matches.id = deliveries.match_id and display 20 joined records.  
SELECT m.id, m.season, m.venue, d.inning, d.batting_team, d.bowling_team,  
       d.`over`, d.ball, d.batsman, d.bowler, d.total_runs  
FROM matches m  
JOIN deliveries d ON m.id = d.match_id  
LIMIT 20;
```

The Results window displays the following data:

	id	season	venue	inning	batting_team	bowling_team	over	ball	batsman	bowler	total_runs
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	1	DA Warner	TS Mills	0	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	2	DA Warner	TS Mills	0	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	3	DA Warner	TS Mills	4	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	4	DA Warner	TS Mills	0	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	5	DA Warner	TS Mills	2	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	6	S Dhawan	TS Mills	0	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	7	S Dhawan	TS Mills	1	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	2	1	S Dhawan	A Choudhary	1	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	2	2	DA Warner	A Choudhary	4	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	2	3	DA Warner	A Choudhary	1	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	2	4	DA Warner	A Choudhary	6	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	2	5	DA Warner	A Choudhary	0	
1	2017	Rajiv Gandhi International Stadium, Uppal	1	Sunrisers Hyderabad	Royal Challengers Bangalore	2	6	MC Harris	A Choudhary	0	

The Output window shows the following messages:

#	Time	Action	Message	Duration / Fetch
193	12:45:50	SELECT dismissal_kind, COUNT(*) AS dismissals FROM deliveries WHERE dismissal_kind IS NOT NULL GR...	9 row(s) returned	0.062 sec / 0.000 sec
194	12:46:22	SELECT m.id, m.season, m.venue, d.inning, d.batting_team, d.bowling_team, d.`over`, d.ball, d.batsman, d...	20 row(s) returned	0.000 sec / 0.000 sec

Q42. Using the joined tables, find the total runs scored in each IPL season.

```
SELECT m.season, SUM(d.total_runs) AS season_runs
FROM matches m JOIN deliveries d ON m.id = d.match_id
GROUP BY m.season ORDER BY m.season;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q42. Using the joined tables, find the total runs scored in each IPL season.
SELECT m.season, SUM(d.total_runs) AS season_runs
FROM matches m JOIN deliveries d ON m.id = d.match_id
GROUP BY m.season ORDER BY m.season;
```

The Results window displays the following data:

season	season_runs
2008	17937
2009	16353
2010	18883
2011	21154
2012	22453
2013	22602
2014	18931
2015	18353
2016	18862
2017	18786
2018	20706
2019	20270

The Action Output window shows the following messages:

#	Time	Action	Message	Duration / Fetch
182	12:39:46	SELECT m.id, m.season, m.venue, d.inning, d.batting_team, d.bowling_team, d.'over', d.ball, d.batsman, d...	20 row(s) returned	0.000 sec / 0.000 sec
183	12:40:09	SELECT m.season, SUM(d.total_runs) AS season_runs FROM matches m JOIN deliveries d ON m.id = d.match...	12 row(s) returned	0.422 sec / 0.000 sec

Q43. Using the joined tables, find the total number of sixes in each season.

```
SELECT m.season, COUNT(*) AS sixes
FROM matches m JOIN deliveries d ON m.id = d.match_id
WHERE d.batsman_runs = 6
GROUP BY m.season ORDER BY m.season;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q43. Using the joined tables, find the total number of sixes in each season.
SELECT m.season, COUNT(*) AS sixes
FROM matches m JOIN deliveries d ON m.id = d.match_id
WHERE d.batsman_runs = 6
GROUP BY m.season ORDER BY m.season;
```

The Results window displays the following data:

season	sixes
2008	623
2009	508
2010	587
2011	639
2012	733
2013	681
2014	715
2015	692
2016	639
2017	706
2018	869
2019	778

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
183	12:40:09	SELECT m.season, SUM(d.total_runs) AS season_runs FROM matches m JOIN deliveries d ON m.id = d.match...	12 row(s) returned	0.422 sec / 0.000 sec
184	12:40:52	SELECT m.season, COUNT(*) AS sixes FROM matches m JOIN deliveries d ON m.id = d.match_id WHERE d....	12 row(s) returned	0.063 sec / 0.000 sec

Q44. Using the joined tables, find the total number of fours in each season.

```
SELECT m.season, COUNT(*) AS fours
FROM matches m JOIN deliveries d ON m.id = d.match_id
WHERE d.batsman_runs = 4
GROUP BY m.season ORDER BY m.season;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Q44. Using the joined tables, find the total number of fours in each season.
SELECT m.season, COUNT(*) AS fours
FROM matches m JOIN deliveries d ON m.id = d.match_id
WHERE d.batsman_runs = 4
GROUP BY m.season ORDER BY m.season;
```

The query results are displayed in the Result Grid, showing the total number of fours for each season from 2008 to 2019.

season	fours
2008	1703
2009	1321
2010	1709
2011	1916
2012	1911
2013	2054
2014	1563
2015	1611
2016	1633
2017	1612
2018	1674
2019	1685

The Output pane shows the execution log with two entries:

#	Time	Action	Message	Duration / Fetch
184	12:40:52	SELECT m.season, COUNT(*) AS fours FROM matches m JOIN deliveries d ON m.id = d.match_id WHERE d...	12 row(s) returned	0.063 sec / 0.000 sec
185	12:41:23	SELECT m.season, COUNT(*) AS fours FROM matches m JOIN deliveries d ON m.id = d.match_id WHERE d...	12 row(s) returned	0.110 sec / 0.000 sec

Q45. Using the joined tables, find the total extra runs in each season.

```
SELECT m.season, SUM(d.extra_runs) AS extra_runs
FROM matches m JOIN deliveries d ON m.id = d.match_id
GROUP BY m.season ORDER BY m.season;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q45. Using the joined tables, find the total extra runs in each season.
SELECT m.season, SUM(d.extra_runs) AS extra_runs
FROM matches m JOIN deliveries d ON m.id = d.match_id
GROUP BY m.season ORDER BY m.season;
```

The Results window displays the following data:

season	extra_runs
2008	1128
2009	977
2010	1129
2011	1226
2012	1131
2013	1115
2014	988
2015	926
2016	899
2017	866
2018	800
2019	819

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
185	12:41:23	SELECT m.season, COUNT(*) AS fours FROM matches m JOIN deliveries d ON m.id = d.match_id WHERE d...	12 row(s) returned	0.110 sec / 0.000 sec
186	12:41:52	SELECT m.season, SUM(d.extra_runs) AS extra_runs FROM matches m JOIN deliveries d ON m.id = d.match_id...	12 row(s) returned	0.454 sec / 0.000 sec

Q46. Using the joined tables, find total runs scored by each batting team in each season.

```
SELECT m.season, d.batting_team, SUM(d.total_runs) AS team_runs
FROM matches m JOIN deliveries d ON m.id = d.match_id
GROUP BY m.season, d.batting_team
ORDER BY m.season, team_runs DESC;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
1  -- Q46. Using the joined tables, find total runs scored by each batting team in each season.
2  SELECT m.season, d.batting_team, SUM(d.total_runs) AS team_runs
3  FROM matches m JOIN deliveries d ON m.id = d.match_id
4  GROUP BY m.season, d.batting_team
5  ORDER BY m.season, team_runs DESC;
```

The Results window displays the following data:

season	batting_team	team_runs
2008	Rajasthan Royals	2601
2008	Chennai Super Kings	2520
2008	Kings XI Punjab	2464
2008	Deccan Chargers	2229
2008	Delhi Daredevils	2118
2008	Mumbai Indians	2080
2008	Royal Challengers Bangalore	1983
2008	Kolkata Knight Riders	1942
2009	Deccan Chargers	2408
2009	Royal Challengers Bangalore	2280
2009	Chennai Super Kings	2231
2009	Delhi Daredevils	2131
2009	Kings XI Punjab	1978

The Output window shows the execution details:

#	Time	Action	Message	Duration / Fetch
186	12:41:52	SELECT m.season, SUM(d.extra_runs) AS extra_runs FROM matches m JOIN deliveries d ON m.id = d.match_id	12 row(s) returned	0.454 sec / 0.000 sec
187	12:42:22	SELECT m.season, d.batting_team, SUM(d.total_runs) AS team_runs FROM matches m JOIN deliveries d ON m.id = d.match_id	100 row(s) returned	0.593 sec / 0.000 sec

Q47. Using the joined tables, find the top 10 run scorers in the 2017 season.

```
SELECT d.batsman, SUM(d.batsman_runs) AS runs_2017
FROM matches m JOIN deliveries d ON m.id = d.match_id
WHERE m.season = 2017
GROUP BY d.batsman ORDER BY runs_2017 DESC LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the query for Q47. The left sidebar shows the 'ipl_db' schema with tables 'deliveries' and 'matches'. The 'Result Grid' tab is active, displaying the top 10 run scorers. The 'Output' tab shows the execution log with two queries: one for team runs and one for individual batsman runs.

batsman	runs_2017
DA Warner	641
G Gambhir	498
S Dhawan	479
SPD Smith	472
SK Raina	442
HM Amla	420
MK Pandey	396
KA Pollard	395
PA Patel	395
RA Tripathi	391

Result 60 x

Output

#	Time	Action	Message	Duration / Fetch
187	12:42:22	SELECT m.season, d.batting_team, SUM(d.total_runs) AS team_runs FROM matches m JOIN deliveries d ON ...	100 row(s) returned	0.593 sec / 0.000 sec
188	12:42:52	SELECT d.batsman, SUM(d.batsman_runs) AS runs_2017 FROM matches m JOIN deliveries d ON m.id = d.ma...	10 row(s) returned	0.063 sec / 0.000 sec

Query Completed

Q48. Using the joined tables, find the top 10 venues by total runs scored.

```
SELECT m.venue, SUM(d.total_runs) AS venue_runs
FROM matches m JOIN deliveries d ON m.id = d.match_id
GROUP BY m.venue ORDER BY venue_runs DESC LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q48. Using the joined tables, find the top 10 venues by total runs scored.
SELECT m.venue, SUM(d.total_runs) AS venue_runs
FROM matches m JOIN deliveries d ON m.id = d.match_id
GROUP BY m.venue ORDER BY venue_runs DESC LIMIT 10;
```

The Results window displays the following table:

venue	venue_runs
Eden Gardens	23857
Wankhede Stadium	23649
M Chinnaswamy Stadium	23267
Feroz Shah Kotla	20896
Rajiv Gandhi International Stadium, Uppal	17093
MA Chidambaram Stadium, Chepauk	15696
Sawai Mansingh Stadium	14440
Punjab Cricket Association Stadium, Mohali	10987
Maharashtra Cricket Association Stadium	6871
Dr DY Patil Sports Academy	4810

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
188	12:42:52	SELECT d batsman, SUM(d batsman_runs) AS runs_2017 FROM matches m JOIN deliveries d ON m.id = d.ma...	10 row(s) returned	0.063 sec / 0.000 sec
189	12:43:21	SELECT m.venue, SUM(d total_runs) AS venue_runs FROM matches m JOIN deliveries d ON m.id = d.match_j...	10 row(s) returned	0.516 sec / 0.000 sec

Q49. Using the joined tables, find the number of recorded dismissals in each season.

```
SELECT m.season, COUNT(d.player_dismissed) AS dismissals
FROM matches m JOIN deliveries d ON m.id = d.match_id
GROUP BY m.season ORDER BY m.season;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Q49. Using the joined tables, find the number of recorded dismissals in each season.
SELECT m.season, COUNT(d.player_dismissed) AS dismissals
FROM matches m JOIN deliveries d ON m.id = d.match_id
GROUP BY m.season ORDER BY m.season;
```

The Results window displays the following data:

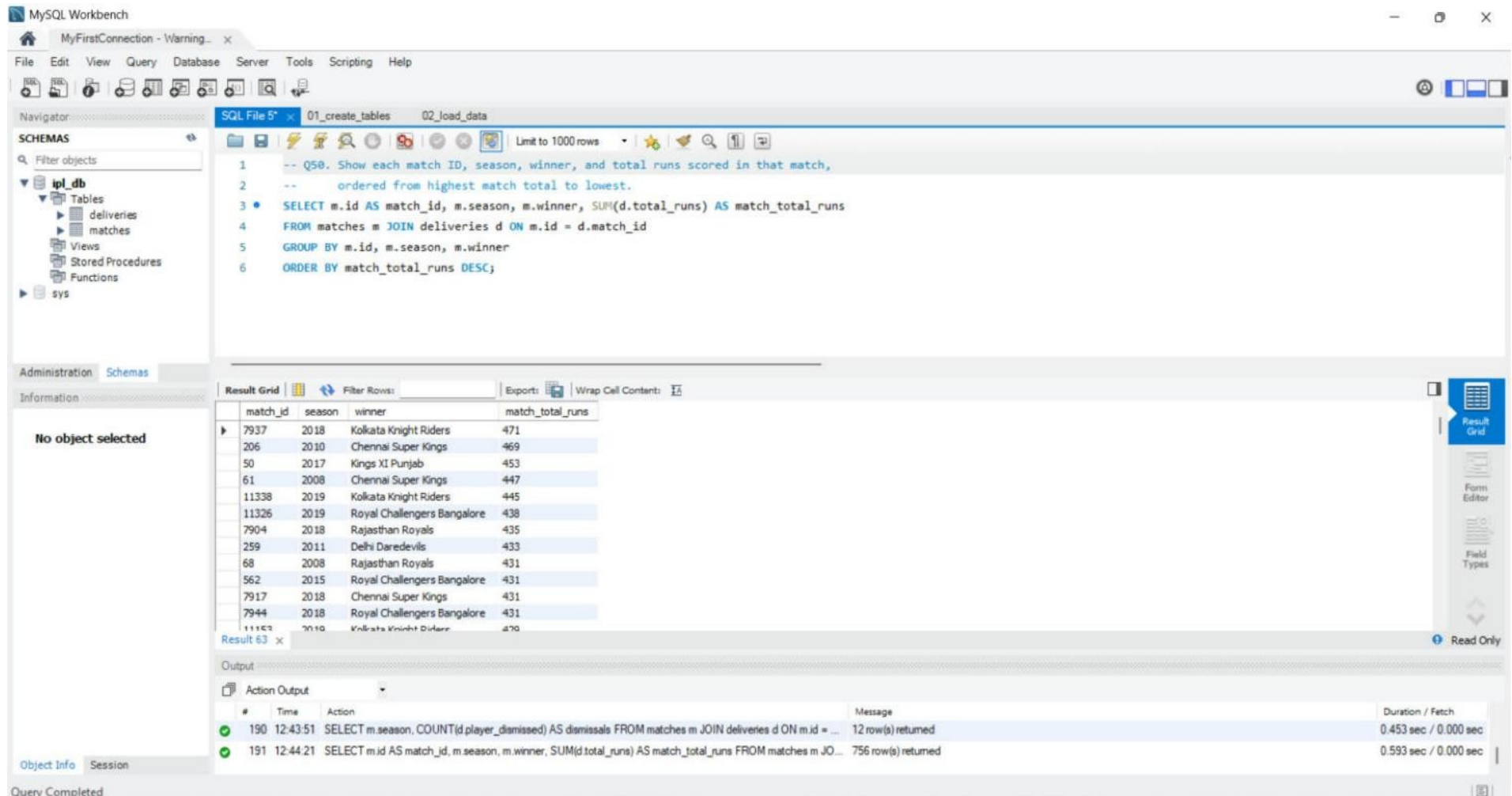
season	dismissals
2008	690
2009	698
2010	725
2011	813
2012	858
2013	912
2014	674
2015	691
2016	666
2017	711
2018	719
2019	677

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
189	12:43:21	SELECT m.venue, SUM(d.total_runs) AS venue_runs FROM matches m JOIN deliveries d ON m.id = d.match_id	10 row(s) returned	0.516 sec / 0.000 sec
190	12:43:51	SELECT m.season, COUNT(d.player_dismissed) AS dismissals FROM matches m JOIN deliveries d ON m.id = ...	12 row(s) returned	0.453 sec / 0.000 sec

Q50. Show each match ID, season, winner, and total runs scored in that match, ordered from highest match total to lowest.

```
SELECT m.id AS match_id, m.season, m.winner, SUM(d.total_runs) AS match_total_runs
FROM matches m JOIN deliveries d ON m.id = d.match_id
GROUP BY m.id, m.season, m.winner
ORDER BY match_total_runs DESC;
```



MySQL Workbench

MyFirstConnection - Warning...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

ipl_db

Tables

deliveries

matches

Views

Stored Procedures

Functions

sys

Administration Schemas

Information

No object selected

SQL File 5* x 01_create_tables 02_load_data

Limit to 1000 rows

```
1 -- Q50. Show each match ID, season, winner, and total runs scored in that match,
2 -- ordered from highest match total to lowest.
3 * SELECT m.id AS match_id, m.season, m.winner, SUM(d.total_runs) AS match_total_runs
4 FROM matches m JOIN deliveries d ON m.id = d.match_id
5 GROUP BY m.id, m.season, m.winner
6 ORDER BY match_total_runs DESC;
```

Result Grid

	match_id	season	winner	match_total_runs
▶	7937	2018	Kolkata Knight Riders	471
	206	2010	Chennai Super Kings	469
	50	2017	Kings XI Punjab	453
	61	2008	Chennai Super Kings	447
	11338	2019	Kolkata Knight Riders	445
	11326	2019	Royal Challengers Bangalore	438
	7904	2018	Rajasthan Royals	435
	259	2011	Delhi Daredevils	433
	68	2008	Rajasthan Royals	431
	562	2015	Royal Challengers Bangalore	431
	7917	2018	Chennai Super Kings	431
	7944	2018	Royal Challengers Bangalore	431
	11152	2010	Kolkata Knight Riders	430

Result 63 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
190	12:43:51	SELECT m.season, COUNT(d.player_dismissed) AS dismissals FROM matches m JOIN deliveries d ON m.id = ...	12 row(s) returned	0.453 sec / 0.000 sec
191	12:44:21	SELECT m.id AS match_id, m.season, m.winner, SUM(d.total_runs) AS match_total_runs FROM matches m JO ...	756 row(s) returned	0.593 sec / 0.000 sec

Object Info Session

Query Completed